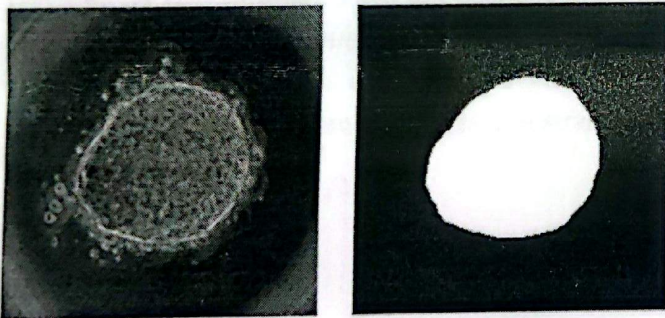


Aluno: _____

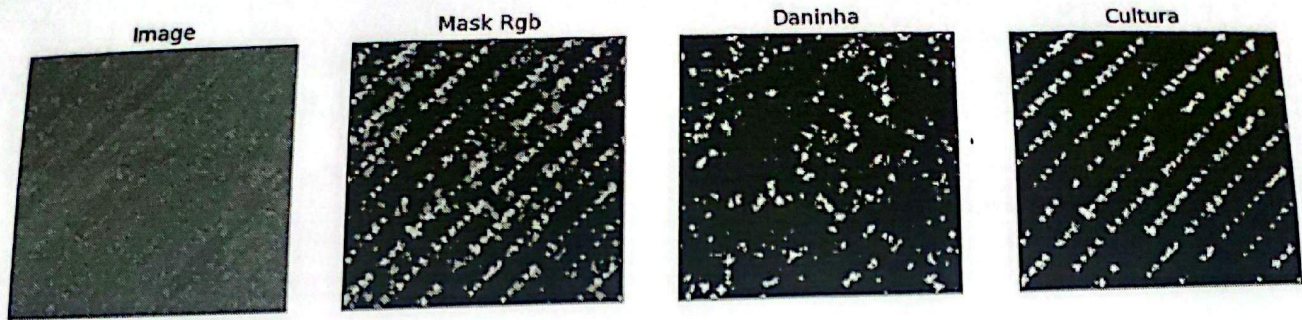
Gabriel

Questão 01 (2.5 pontos). Faça uma função em pseudocódigo de baixo nível (com varredura da matricial valor por valor) que dada uma pilha com P mapas de características $N \times M$ (que pode ser considerada uma matriz $N \times M \times P$), calcule *max pooling* com um tamanho de filtro $s \times s$ e deslizamento (*stride*) de valor s .

Questão 02 (2.5 pontos). Faça uma função em pseudocódigo a nível de laços *for* (para), que dados uma imagem original e uma máscara binária correspondente a segmentação de um objeto da imagem original, calcule a caixa delimitadora mínima representada por (x_0, y_0, h, w) , em que (x_0, y_0) são as coordenadas do topo superior esquerdo da caixa, h é a altura da caixa e w é a largura. Em seguida, use a caixa delimitadora para extrair o objeto da imagem original. A Figura abaixo ilustra uma possível imagem original (à esquerda) e a máscara binária com o objeto detectado (à direita).



Questão 03 (2.5 pontos). Redes neurais de segmentação semântica como a U-net decompõe as anotações de múltiplas classes de objetos de uma imagem em uma pilha de máscaras binárias, sendo uma máscara binária por classe de objeto. Considere uma imagem original de um plantio e a anotação desta (máscara RGB) marcando com rótulo $[255, 0, 0]$ os pixels das ervas daninhas e rótulo $[0, 255, 0]$ os pixels da cultura agrícola. Faça uma função em pseudocódigo baixo nível, que dada a máscara de anotação colorida gere as máscaras binárias para cultura e erva-daninha conforme ilustra a figura a seguir.



Questão 04 (2.5 pontos). Faça um pseudocódigo de baixo nível, varrendo as matrizes por laços *for* que dada uma matriz $N \times M \times P$ de entrada é um filtro $R \times S \times P$, sendo R e P valores inteiros ímpares tal que $R \ll N$ e $S \ll M$. O sinal $\ll$ denota “muito menor”. Seu código deve funcionar para qualquer valor inteiro de P , com $P \geq 1$.

Questão 05 (2.5 pontos). Faça um pseudocódigo de baixo nível, varrendo a matriz de imagem por laços *for*, que dada uma matriz imagem com 256 níveis de cinza, calcule a matriz de coocorrência.

Questão 06 (2.5 pontos). Faça um pseudocódigo de baixo nível, varrendo as matrizes por laços *for* que calcule a medida de *Intersection over Union* (IoU). Essa medida normalmente aplica para medir a performance de métodos de detecção/segmentação de objetos sendo dada pela fórmula abaixo. A Figura dada em seguida também ajuda a ilustrar a medida de IoU.

$$IoU = \frac{A \cap B}{A \cup B}$$

alternativamente IoU pode ser calculado por:

$$IoU = \frac{TP}{FN + TP + FP},$$

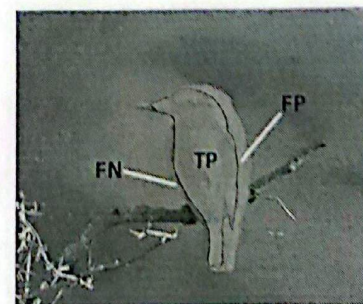
onde TP é a quantidade de verdadeiros positivos, ou seja, pixels do objeto que foram preditos como sendo do objeto, FN é a quantidade de falsos negativos, ou seja, pixels do objeto que não foram detectados e FP é a quantidade de falsos positivos, ou seja pixels que foram preditos como sendo do objeto não na verdade não pertencem ao objeto.



(A) ground-truth



(B) predição



(C) TP, FN, FP

Questão 01

```

func max_pooling(pmapas, s)
    N, M, P = size(pmapas)
    para k = 0 até P-1
        para i = 0 até (N//s)-1
            para j = 0 até (M//s)-1
                maximo = 0
                para x = i*s até (i*s)+s-1
                    para y = j*s até (j*s)+s-1
                        se pmapas[x, y, k] > maximo
                            maximo = pmapas[x, y, k]
                pmapas_res[i, j, k] = maximo
    retorna pmapas_res

```

comentário
// calcula a divisão inteira

Questão 02

```

func extract_bbox(Iorig, Ibin)
    N, M = size(Ibin)
    x0 = ∞; xf = -∞; y0 = ∞; yf = -∞
    para i = 0 até N-1
        para j = 0 até M-1
            se Ibin[i, j] == 1
                se i < x0
                    x0 = i
                se j < y0
                    y0 = j
                se i > xf
                    xf = i
                se j > yf
                    yf = j
    h = xf - x0
    w = yf - y0
    # Agora temos x0, y0, h e w, e extraímos o objeto
    para x = 0 até h-1
        para y = 0 até w-1
            bbox[x, y] = Iorig[x0+x, y0+y]
    retorna bbox

```

Questão 03

```

func gera_masc_bin(masc_cor)
    N, M, _ = size(masc_cor)
    ERVA[1:N, 1:M] = 0
    CULT[1:N, 1:M] = 0
    para i = 0 até N-1
        para j = 0 até M-1
            se masc_cor
                ERVA[i, j] = 1
            se masc_cor[i, j] = [0, 255, 0]
                CULT[i, j] = 1
    retorne ERVA, CULT
    
```

Questão 04

```

# Entradas:
# matriz: matriz de dimensão  $N \times M \times P$ 
# filtro: matriz de dimensão  $R \times S \times P$ 
#  $N, M, P, R, S$ : inteiros com  $R$  e  $S$  ímpares ( $R \ll N, S \ll M$ )
# Dimensões do resultado da convolução sem padding
h = N - R + 1
w = M - S + 1
# Inicializar matriz resultado com zeros
res[1:h, 1:w] = 0
para x = 0 até h
    para y = 0 até w
        para fx = 0 até R-1
            para fy = 0 até S-1
                para p = 0 até P-1
                    res[x, y] += matriz[x+fx, y+fy, p] * filtro[fx, fy, p]
# res guardará o resultado
    
```


Questão 05
Entradas:

img: imagem em níveis de cinza
dx: inteiro representando o deslocamento na linha
dy: " " " " na coluna

$N, M = \text{size}(img)$

$cooc[0:256, 0:256] = 0$

para $x = 0$ até $N-1$

para $y = 0$ até $M-1$

se $x+dx \geq 0$ AND $x+dx < N$ AND $y+dy \geq 0$ AND $y+dy < M$

$cooc[img[x,y], img[x+dx, y+dy]] += 1$

Questão 06

Entradas:

img_gt: máscara de ground-truth

img_pred: máscara de predição

N, M : dimensões, que são iguais para ambos img_gt e img_pred

contIntersec = 0

contUnion = 0

para $x = 0$ até $N-1$

para $y = 0$ até $M-1$

$contUnion += img_pred[x,y] \text{ OR } img_gt[x,y]$

$contIntersec += img_pred[x,y] \text{ AND } img_gt[x,y]$

$IoU = contUnion / contIntersec$